CALIDAD DE SOFTWARE

Automatización de Pruebas

Capítulo 6

M en C JESUS MARTÍN SILVA FERNÁNDEZ
jsilvaf@unsa.edu.pe

Agenda

1	Actividades de prueba/entregables del SDLC
2	Modelos y marcos de automatización
3	Incorporación de la automatización en los procesos de prueba estándar
4	Profundización en las pruebas de artefactos
5	Estudio de caso
6	Resumen de la presentación

Actividades de prueba del SDLC/resultados

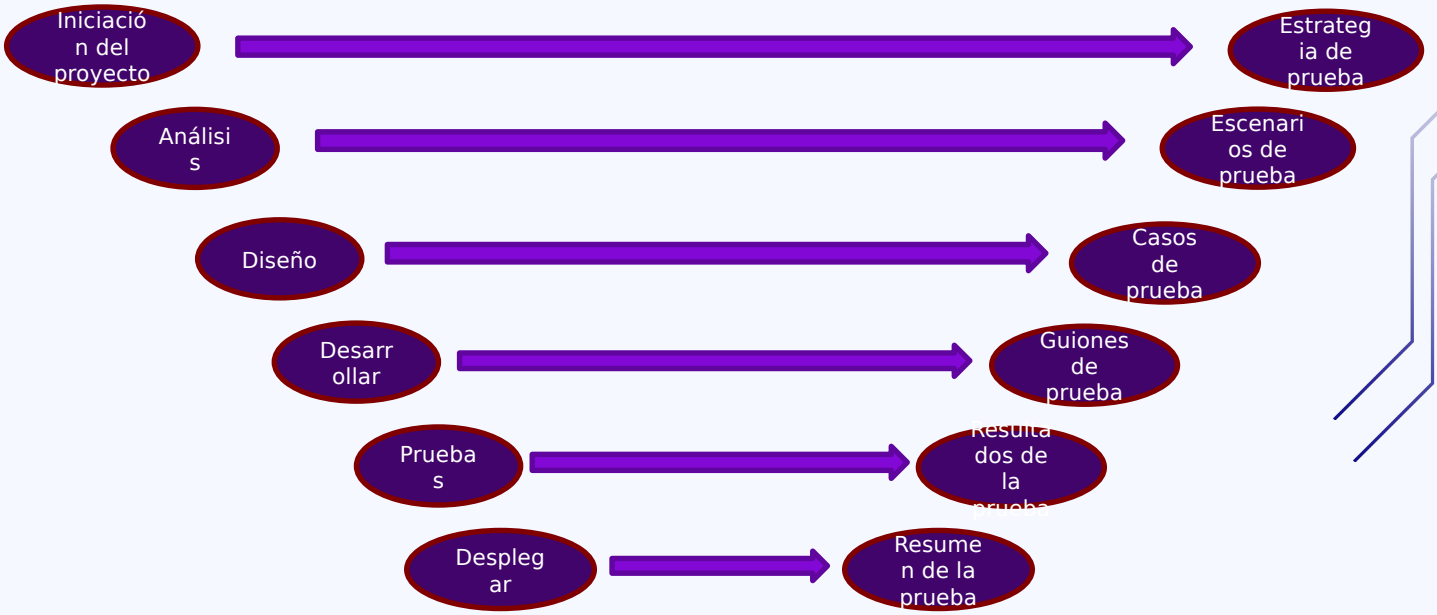
Actividades de prueba dentro de un SDLC ágil

Proceso típico ágil de actividades a entregables

Iniciación del proyecto	Obtenga una comprensión del proyecto
Planificación de lanzamiento	Participar en el dimensionamiento de historias; Crear planes de prueba
Cada iteración 1  … Incógnita	Participar en la planificación del sprint y estimar tareas. Escribir y ejecutar pruebas de historias. Realice pruebas en pares con otros evaluadores y desarrolladores. Validación de negocio (clientes) Automatizar nuevos casos de prueba funcionales; ejecutar casos de prueba de regresión automatizados Ejecutar pruebas de carga del proyecto Demostración a las partes interesadas
El juego final (Prueba del sistema)	La administración de versiones prueba la implementación simulada en el entorno de pruebas; prueba de humo en el entorno de pruebas Realizar prueba de carga (si es necesario) Prueba de regresión completa; Participar en la preparación de
Lanzamiento a producción/soporte	Participar en el lanzamiento a producción Participar en retrospectivas

Actividades de prueba dentro de un SDLC en cascada

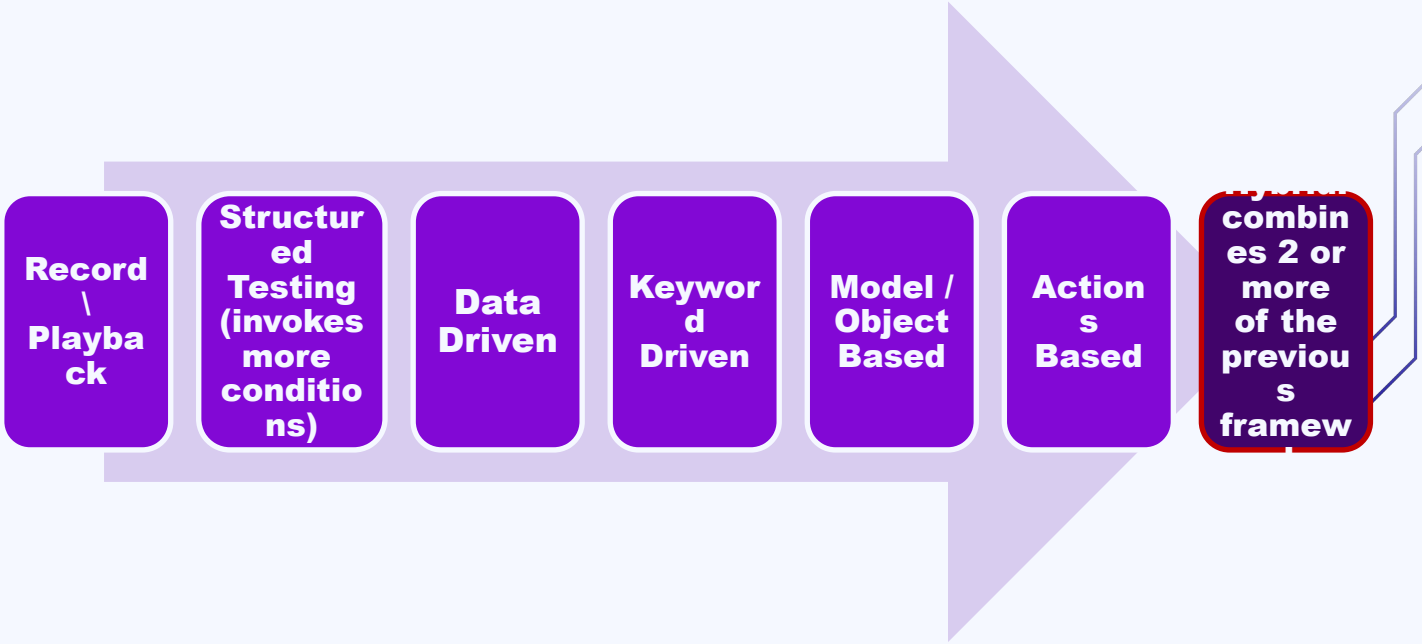
Conjunto de actividades de prueba del modelo V estándar



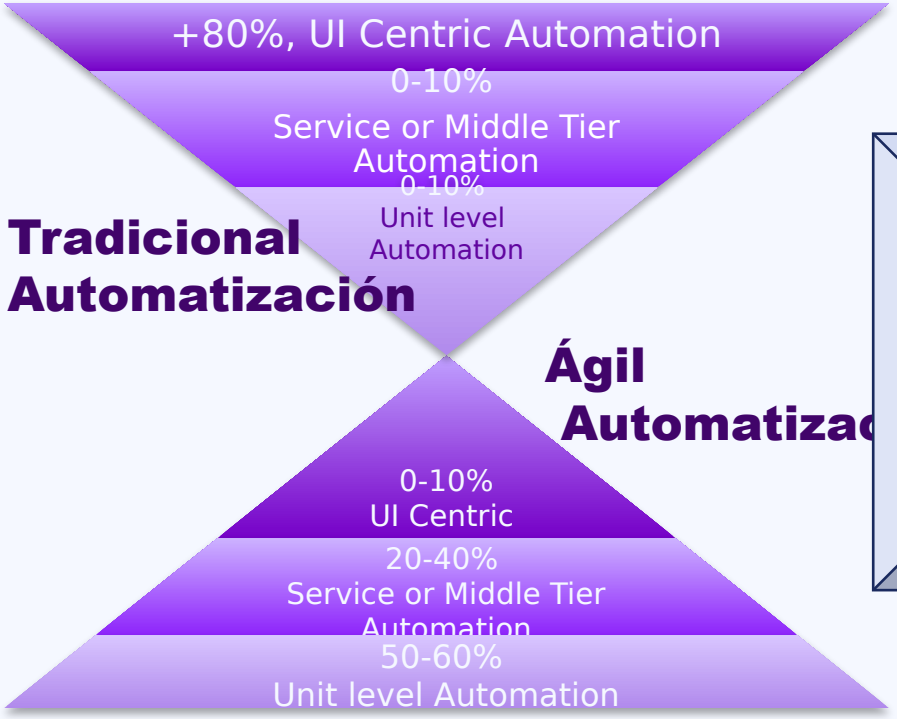
Modelos y marcos de trabajo de automatización de pruebas en la industria

Marcos de automatización de pruebas

A continuación se muestra la evolución de los marcos de automatización.



Puntos clave con la automatización de pruebas y Agile



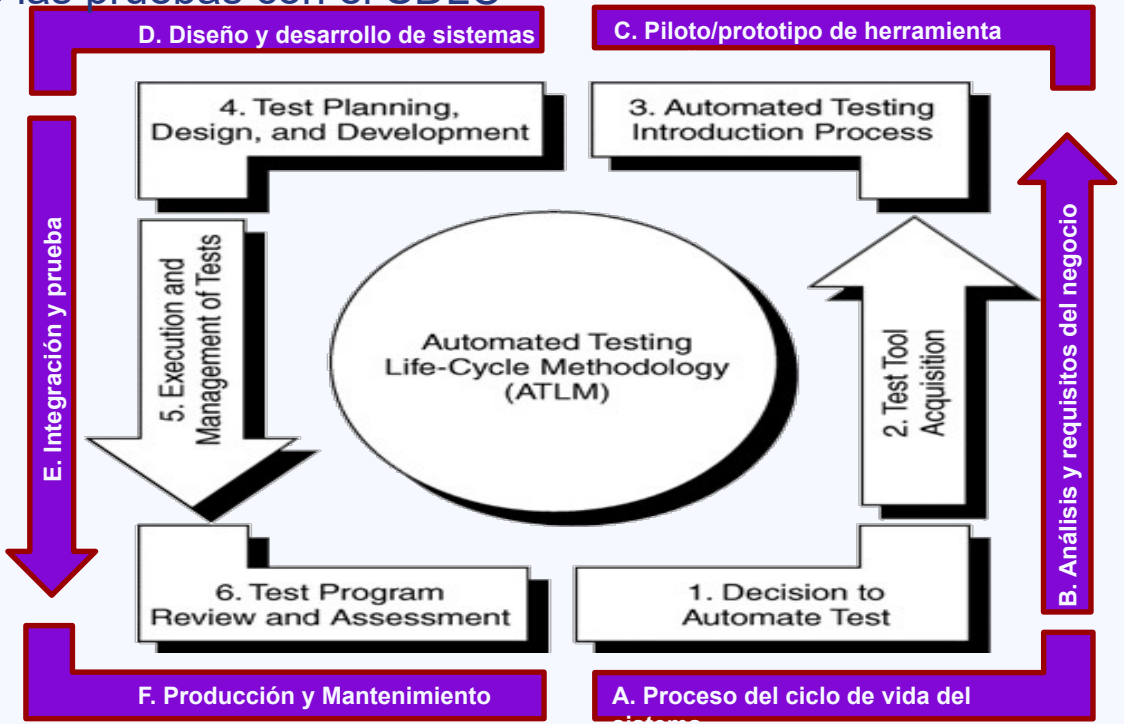
Automatización ágil

- Unidades de trabajo más pequeñas construidas de forma única en comparación con unidades de trabajo de principio a fin
- Revisión constante del backlog de automatización
- Automatización de formularios no completos

Velocity Partners: la pirámide de automatización de pruebas ágiles, Mike Cohn

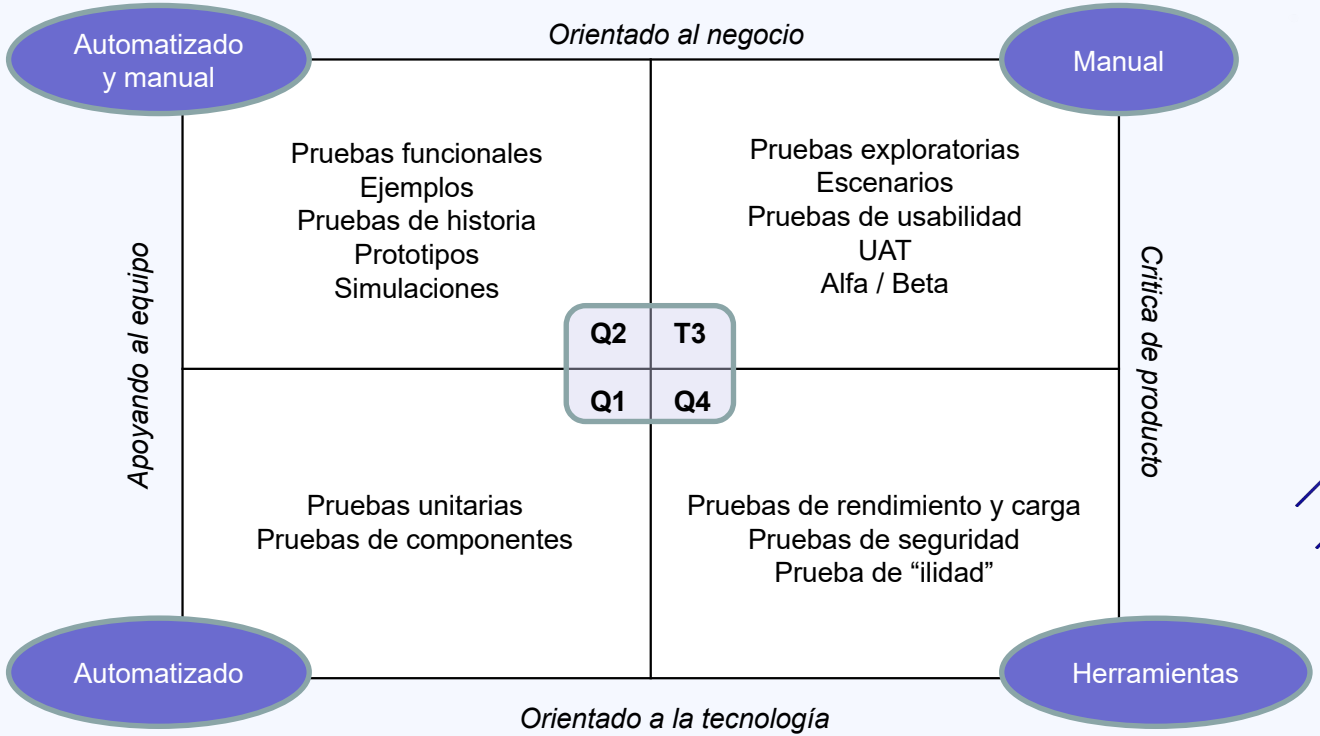
Mapeo de ATLM contra un SDLC en cascada

Cómo se relaciona la gestión automatizada del ciclo de vida de las pruebas con el SDLC



Incorporando la automatización al proceso

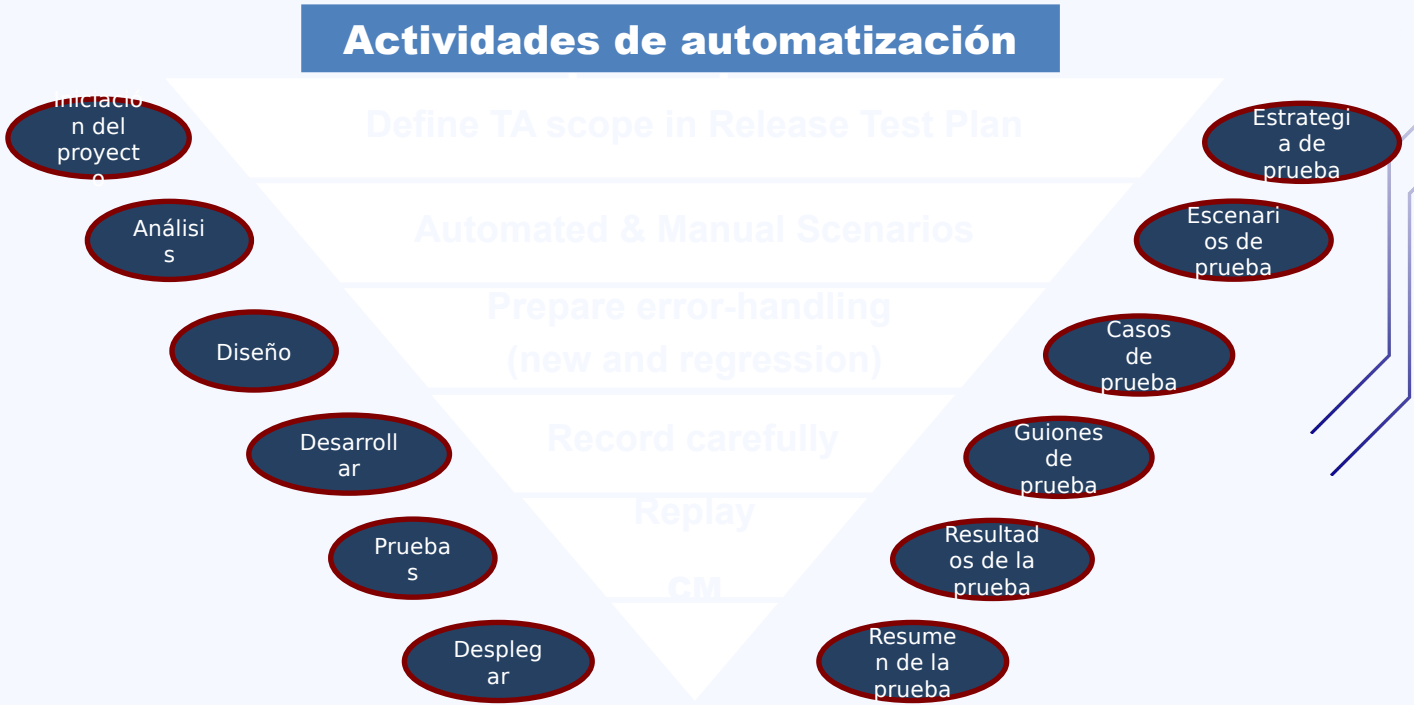
Automatización de pruebas: fusión ágil



Pruebas ágiles: Crispin y Gregory

Automatización de pruebas – Agile (eso es) fusión

Ya nadie admite hacer Cascada pura



Profundización en las pruebas de artefactos

Estrategia de prueba / Plan de prueba

Evitar

Omitting automation

ting nation for complishing the testing on schedule

Omitting variables:

Resources, Data, Objects, Licenses, Environments

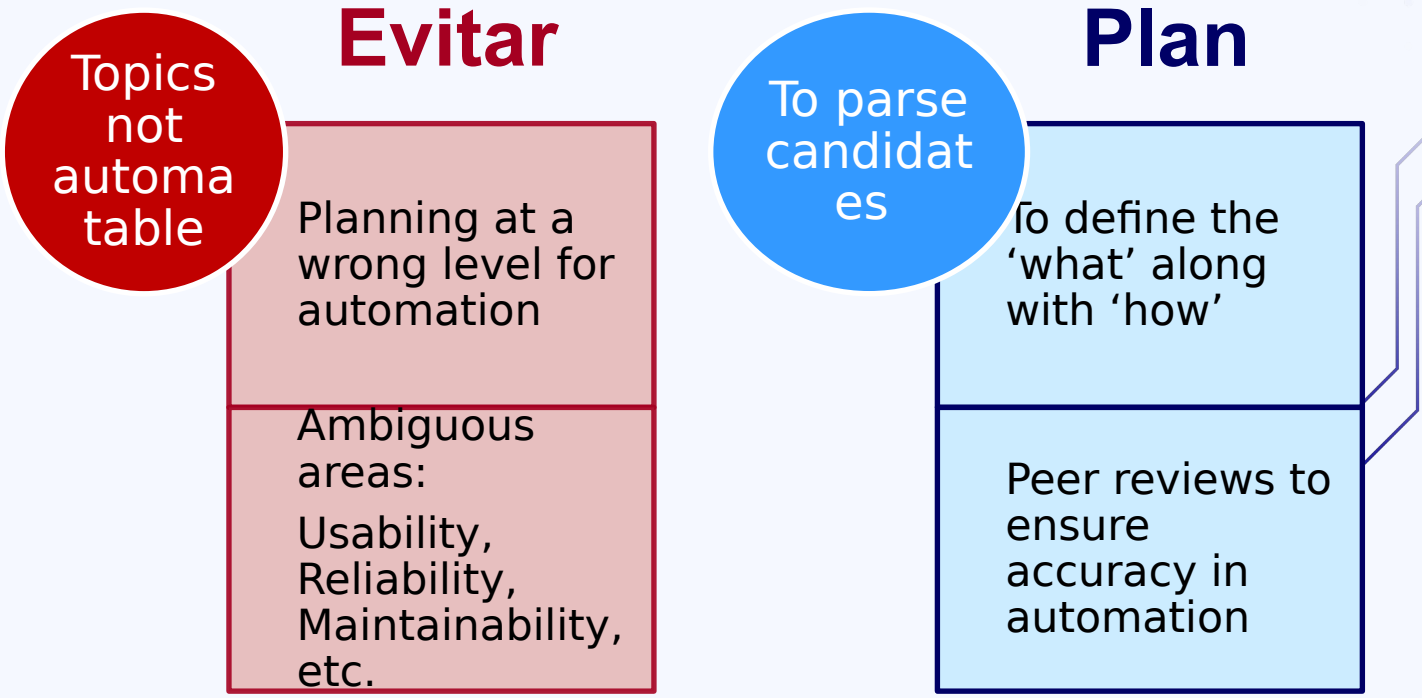
Plan

To obtain buy-in for automation

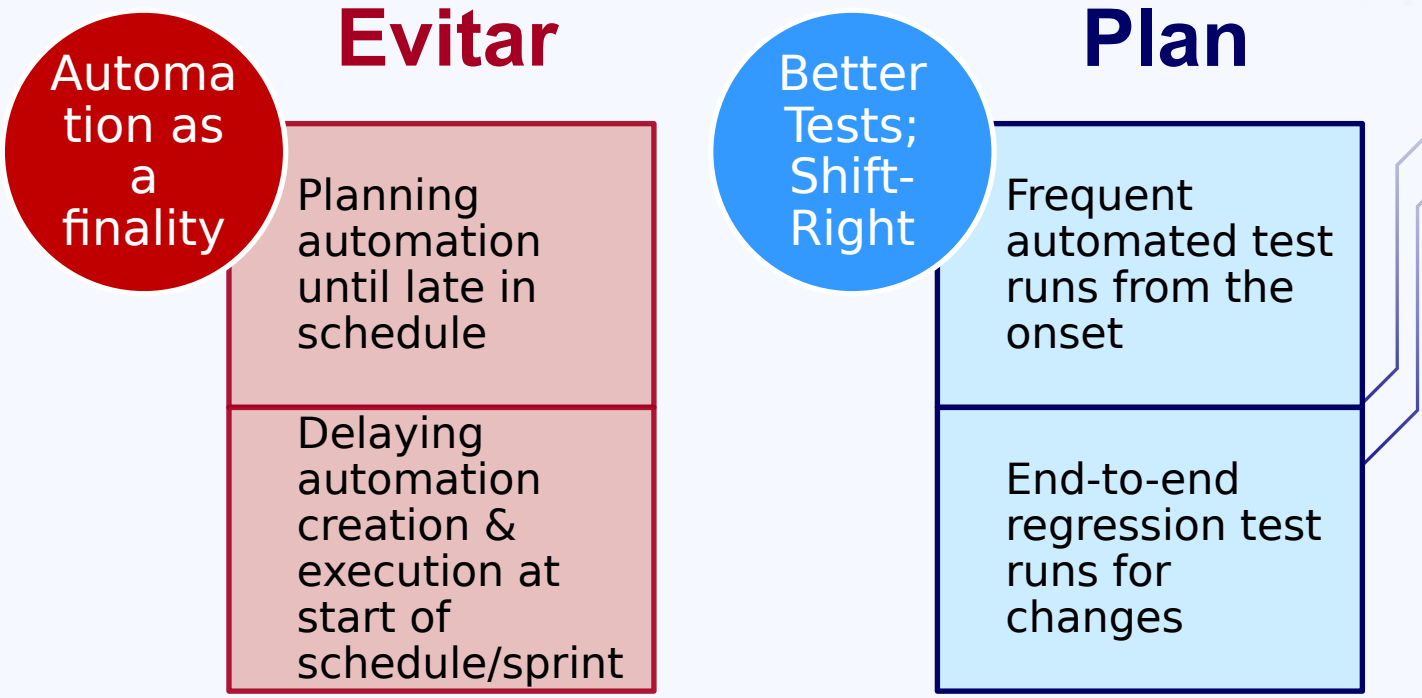
tool n was and how testers will use this

To include automation in every iteration / build

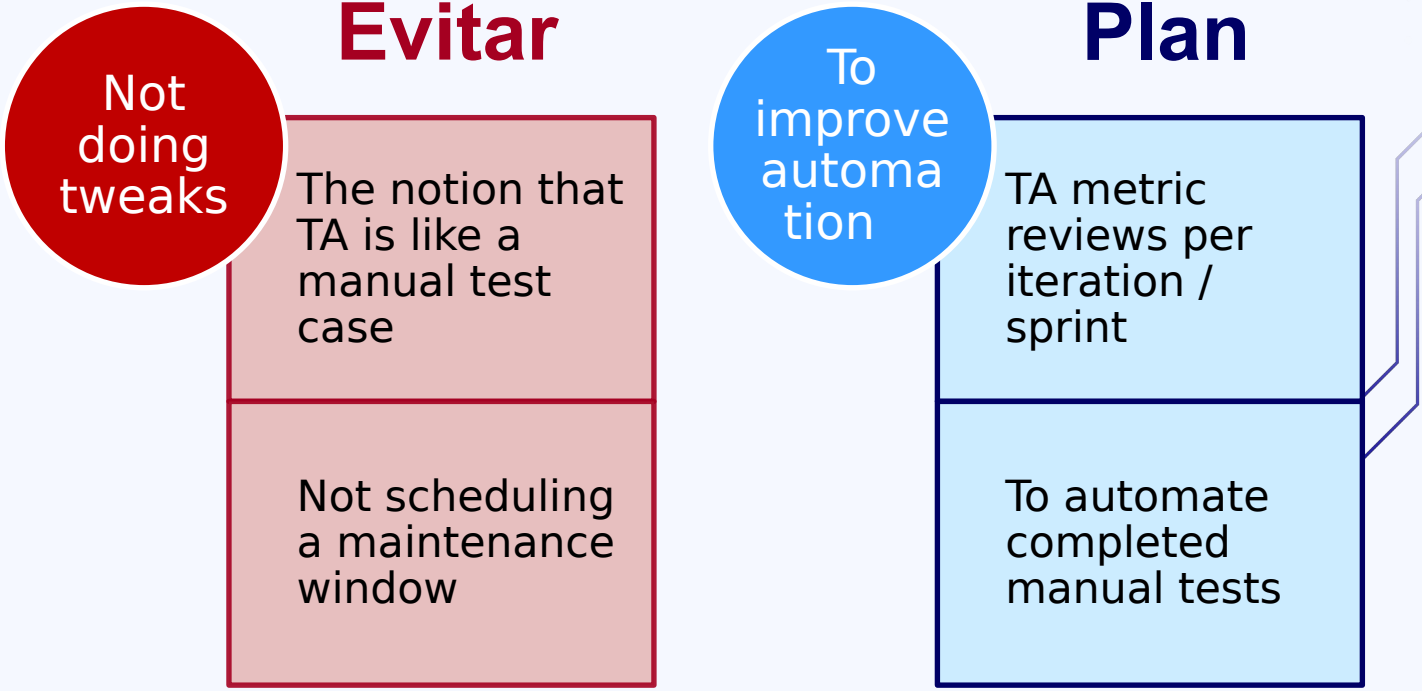
Historias de prueba / Escenarios de prueba



Guiones de prueba/Casos de prueba



Retrospectivas / Resumen de pruebas



Estudio de caso

Antecedentes y solución

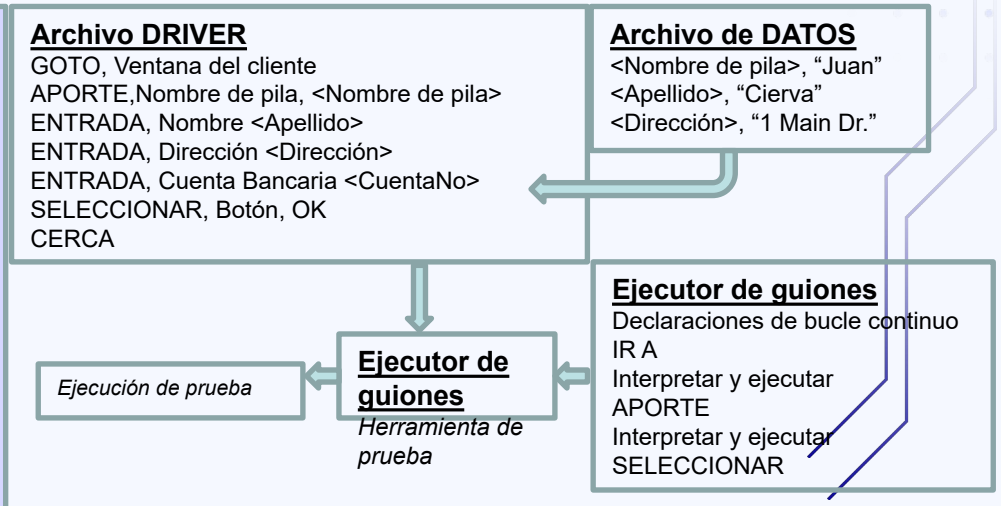
Titulado: Automatización a través de la puerta trasera (soporte de pruebas manuales)

Fondo:
Para mejorar la tasa de automatización, se requirieron cambios en el marco de TA para admitir pruebas manuales.

Solución técnica:
Desarrollar un marco denominado prueba basada en comandos

¿Qué son las pruebas basadas en comandos?

- Utiliza palabras clave de comando simples (SELECT, BUTTON)
- Los scripts de interpretación son los mismos para todos los productos.
- Utilizando las ventajas de las pruebas basadas en datos, la navegación estaba en un archivo DRIVER; los datos en un archivo DATA.
- Dos archivos creados juntos forman un script de comando
- De forma secuencial, el ejecutor de scripts lee los comandos en el archivo DRIVER. Los códigos DATA se sustituyen por datos del archivo DATA.



Con pruebas basadas en comandos:

- Los evaluadores no necesitan aprender a usar scripts de herramientas
- La separación en la navegación y los datos hace que los scripts de comandos sean flexibles y reutilizables
- Los archivos DRIVER se pueden portar a diferentes aplicaciones y pueden migrar a otra herramienta
- La herramienta de prueba solo es necesaria para preparar las plantillas y ejecutar las pruebas.

Proceso de Implementación

Prerrequisitos:

1) No poderempezar sinConocimiento de la aplicación en prueba y de los casos de prueba que se van a automatizar
2) Se necesita un motor que funcione para interpretar los comandos para todos los casos de prueba.
3) Todos los elementos de la GUI utilizados en la ejecución deben ser registrados por la herramienta de captura/reproducción para la normalización del nombre del objeto.

Prerrequisitos

1. **Casos de prueba**
2. **Biblioteca de comandos**
3. **Registro de controles GUI**

PASO1
Automator →Capturar→Script de herramienta

PASO 2
Guión de la herramienta ↓
Máquina ↓
Script de comando

PASO 3
Comando Script →Tester→Suite

Prueba automatizada

Suite

↓

Motor de prueba

↓

Resultados de la prueba

Step 1
Record the TC with the capture functionality of automation solution

Step 2
Translate generated script to command-script

Step 3
Develop TC's in template and build test suites

Arquitectura del playwright

- Enlaces de idioma

- Protocolo de automatización único
- Unificar todos los protocolos de depuración

- Protocolos de depuración de navegadores (nativos),

por ejemplo, el protocolo ChromeDevTools,

Inspector web (Safari)

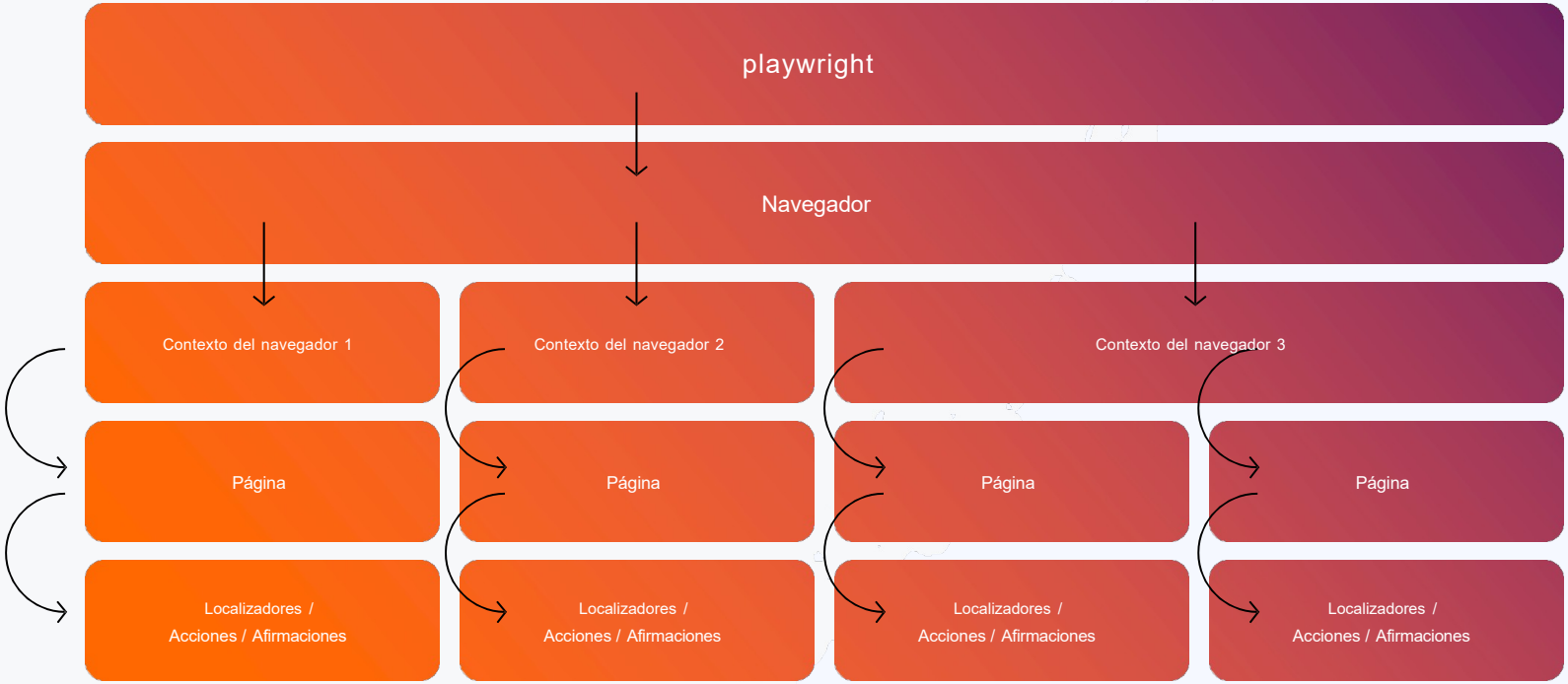


¿Cómo crear una prueba? (1/2)



- El navegador solo necesita inicializarse una vez
- Uso compartido de navegadores utilizando aislados BrowserContext es posible y rápido
- Cada BrowserContext tiene su propio caché y cookies

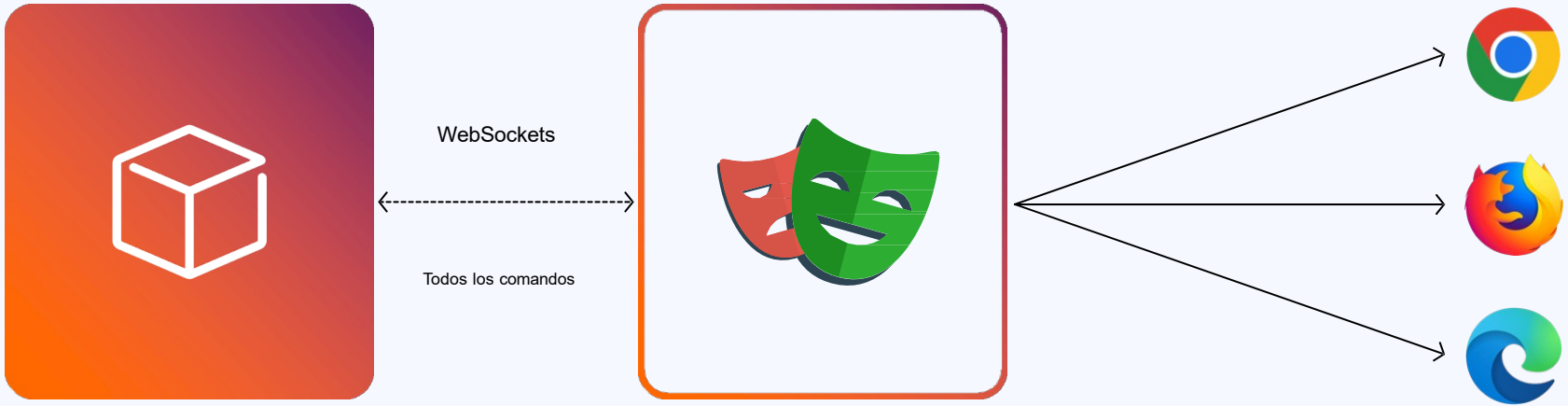
¿Cómo crear una prueba? (2/2)



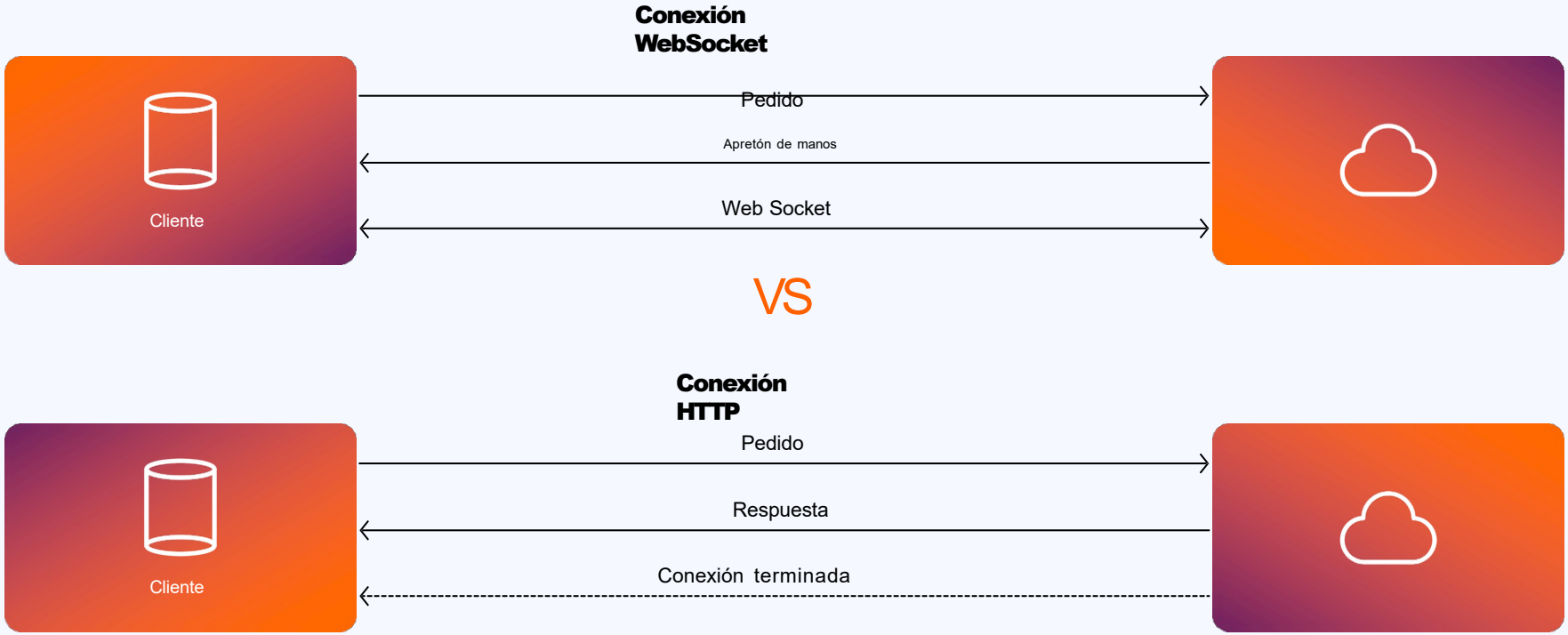
Selenio


Java
python
{JSON}
HTTP terminado
Servidor
HTTP
HTTP terminado
Servidor
HTTP
Protocolo de cable
a través de HTTP
Selenio
Biblioteca de clientes
Navegadores
Conductor
Navegadores

playwright



playwright vs. Selenium



Proceso estándar de pruebas

Etapa	Actividad principal	Evidencia
Planificación	Definir alcance, riesgos y criterios	Plan y requisitos
Diseño	Precisar datos, pasos y oráculos	Casos y trazabilidad
Implementación	Separar acciones, datos y aserciones	Suite versionada
Ejecución	Aislar sesiones y repetir pruebas	Reporte y trazas
Análisis y cierre	Clasificar fallos y decidir	Incidentes y conclusión

El equipo revisa y mantiene la suite durante cada iteración del desarrollo.

Resumen de normas y estándares

Referencia	Aplicación
ISO/IEC/IEEE 29119 partes 1–4	Conceptos 2022, procesos, documentación y técnicas 2021
ISO/IEC/IEEE 29119-5:2024	Automatización dirigida por palabras clave
ISO/IEC 25010:2023	Características para evaluar calidad del producto
ISO/IEC 5055:2021	Medición automatizada de calidad del código fuente
W3C WebDriver	Interfaz estándar de automatización del navegador

URLs oficiales en las notas. ISO e IEEE pueden requerir compra o acceso institucional.

Caso de implementación con Playwright

Aplicación pública de demostración

https://www.saucedemo.com/

Preparación del proyecto

```
npm init playwright@latest
npx playwright install chromium
```

Archivo tests/automatizacion.spec.js

Importar test y expect. Usar la función login(page, clave) de las notas.
Cada prueba inicia con una sesión independiente.

Prueba 1 de acceso válido

```
test('T01 acceso válido', async ({ page }) => {
  await login(page);
  await expect(page).toHaveURL(/inventory\.html$/);
  await expect(
    page.locator('[data-test=inventory-list]')
  ).toBeVisible();
});
```

Oráculo: el usuario llega al inventario y puede ver la lista de productos.

Prueba 2 de contraseña incorrecta

```
test('T02 clave incorrecta', async ({ page }) => {
  await login(page, 'clave_incorrecta');
  await expect(
    page.locator('[data-test=error]')
  ).toContainText('Username and password do not match');
  await expect(
    page.locator('[data-test=login-button]')
  ).toBeVisible();
  await expect(page).not.toHaveURL(/inventory\.html$/);
});
```

Oráculo: mensaje de rechazo, formulario visible y ausencia de acceso al inventario.

Prueba 3 de agregar y retirar del carrito

```
test('T03 carrito', async ({ page }) => {
  await login(page);
  const badge = page.locator('[data-test=shopping-cart-badge]');
  await page.locator(
    '[data-test=add-to-cart-sauce-labs-backpack]'
  ).click();
  await expect(badge).toHaveText('1');
  await page.locator(
    '[data-test=remove-sauce-labs-backpack]'
  ).click();
  await expect(badge).toHaveCount(0);
});
```

Oráculo: el contador indica 1 al agregar y desaparece al retirar el producto.

Criterios de aceptabilidad

Criterio docente	Condición para aceptar el alcance
Corrección funcional	T01, T02 y T03 cumplen todas sus aserciones
Repetibilidad	Tres pasadas sin fallos ni reintentos automáticos
Trazabilidad	Cada requisito enlaza con un caso, script y reporte
Incidentes	Ningún fallo crítico sin explicación o resolución
Reproducción	Versiones, dependencias y comandos disponibles

Un error esperado puede aprobar la prueba. La decisión solo cubre los tres comportamientos evaluados.

Generación de evidencias

```
import { defineConfig } from '@playwright/test';
export default defineConfig({
  testDir: './tests', workers: 1, retries: 0,
  reporter: [ ['html', { open: 'never' }],
              ['junit', { outputFile: 'resultados.xml' }] ],
  use: {
    baseURL: 'https://www.saucedemo.com',
    browserName: 'chromium',
    trace: 'on', screenshot: 'only-on-failure'
  }
});
```

Reporte HTML para revisión, XML para resumen automático y trazas para diagnóstico.

Evidencias y análisis de resultados

Ejecución y consulta

```
npx playwright test
npx playwright show-report
```

Archivo por pasada

Conservar reporte completo, XML y test-results en corrida-01, corrida-02 y corrida-03.

Registro mínimo

ID de caso, fecha, versión, navegador, resultado, duración y ruta de evidencia.

Diagnóstico y decisión

Clasificar cada fallo: producto, script, datos o entorno. Explicar el impacto y decidir.

Caso práctico: puntos clave

- También se adoptó el marco controlado por comandos para pruebas manuales.
- Ayudó a limitar el trabajo de los evaluadores en la misma plantilla simultáneamente
- Informar defectos era mucho más fácil con el retrabajo para recrear el problema.
- Revisiones constantes para evaluar las necesidades actuales de pruebas automatizadas y manuales.
- La implementación admitió la ejecución de pruebas parcialmente automatizadas; para tareas de preparación de pruebas
- Las pruebas manuales ahora se centran en las condiciones específicas del cliente.

Control de Lectura

Revisar y responder:
https://assets.website-files.com/5eb9845c1d37d7efad64e226/61ccd7678d42bd2f8a545de8_eBook-Getting-started-with-test-automation.pdf

1. En el ciclo de vida de Prueba, cual actividad es la de mayor complejidad y costo?
2. Indicar tres de las razones mas importantes para automatizar las pruebas?. Cual es el criterio de selección
3. Cual es el alcance de la herramienta Selenium para automatización de pruebas, identificar características faltantes

PRESENTACION
Individual
Responder preguntas
Fecha: Sesión actual



Referencias

1. https://umu.diva-portal.org/smash/get/diva2:1850362/FULLTEXT01.pdf
2. https://downloads.ctfassets.net/63kg6hcp5guk/1fFESw53ZrIiOT88htjASF/4506805e0e36078f667d2ed4f1e05554/2021-04-26_Automated_testing_Devbridge.pdf

Trabajo Parcial 6:
Aplicación de norma ISO/IEC/IEEE 29119

Revisar:
trab_6.pdf

1. Revisar e implementar documento compartido.
2. Generar entregables.

Grupo: 05 personas
Entregable: Reporte y presentación ppt
Entrega: 20/11/2025

Thanks!

Do you have any questions?

jsilvaf@unsa.edu.pe





Calidad de Software